Trabalho Prático #01 -
KNN

Integrantes:

- Gilmar Silva de Medeiros Filho - 202320945
- Samuel de Oliveira Vanoni - 202320611

1. Introdução e Metodologia

O algoritmo K-Nearest Neighbors (KNN) é um método fundamental de classificação baseado em instâncias. O presente trabalho tem como objetivo solidificar o entendimento deste algoritmo através de sua implementação "*hardcore*" em Python , aplicando-o na classificação da base de dados Iris. Para fins de validação e análise crítica, o classificador construído do zero é comparado diretamente com o algoritmo correspondente da biblioteca ***Scikit-Learn***. A metodologia de avaliação consistiu em rodar ambos os modelos variando o número de vizinhos k = { 1,3,5,7 } e extrair suas respectivas matrizes de confusão. A partir dessas matrizes, foram calculadas e comparadas as métricas de acurácia, precisão e revocação, permitindo mensurar a viabilidade e a exatidão da implementação manual frente a uma solução de mercado.

2. Resultados: Avaliação e Métricas

Para avaliar o desempenho de ambas as abordagens, os classificadores foram submetidos ao conjunto de testes variando os hiperparâmetros k = { 1,3,5,7 }. A Tabela 1 sumariza a comparação das métricas de Acurácia, Precisão e Revocação entre a implementação manual (Hardcore) e a biblioteca Sklearn.

Valor de *k*	**Modelo**	**Acurácia**	**Precisão (Macro)**	**Revocação (Macro)**
$k = 1$	Hardcore	0,93	0,94	0,93
	Sklearn	0,93	0,94	0,93
$k = 3$	Hardcore	0,90	0,90	0,90
	Sklearn	0,90	0,90	0,90
$k = 5$	Hardcore	0,90	0,90	0,90
	Sklearn	0,90	0,90	0,90
$k = 7$	Hardcore	0,93	0,93	0,93
	Sklearn	0,93	0,93	0,93

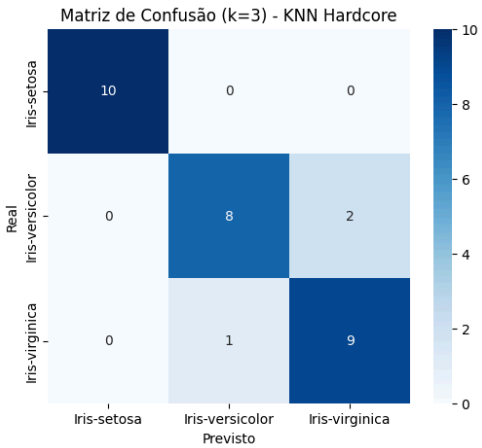
Observa-se que ambas as implementações obtiveram resultados idênticos. O melhor desempenho geral foi alcançado com $k = 1$, onde o modelo atingiu acurácia de 93% e precisão

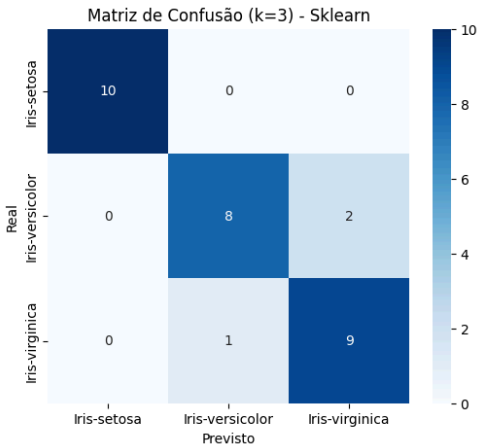
de 94%, porém isso pode ser devido ao overfitting relacionado à escolha de apenas 1 vizinho para comparação onde o modelo decora os padrões dos dados ao invés de aprender e predizer. A precisão e a acurácia mantiveram-se altas, indicando que os modelos possuem baixos índices de falsos positivos e falsos negativos.

Análise das Matrizes de Confusão:

A análise das matrizes de confusão para os diferentes valores de *k* revela que as classes Iris Setosa foram classificadas perfeitamente em quase todos os cenários, devido à sua separação linear natural. As raras confusões (erros de classificação) concentraram-se na fronteira de decisão entre as classes Iris Versicolor e Iris Virginica, um comportamento esperado para este dataset.

Matrizes de Confusão:





3. Análise de Desempenho Computacional

Como as métricas de acurácia de ambos os modelos foram idênticas (comprovando a eficácia matemática que foi desenvolvida), o diferencial dos dois métodos está no tempo de execução, onde houve uma grande diferença que fica evidenciada na tabela a seguir:

Valor de k	KNN Hardcore (ms)	KNN Sklearn (ms)
$k = 1$	542,13	7,51
$k = 3$	507,69	7,19
$k = 5$	522,71	3,93

$k = 7$	594,88	4,50

Os resultados evidenciam uma disparidade expressiva: a implementação Hardcore consumiu em média 541,85 ms por execução, enquanto o modelo da biblioteca Sklearn obteve uma média de 5,78 ms (sendo cerca de 94 vezes mais rápida).

Essa diferença brutal de desempenho não indica um erro na lógica do nosso algoritmo, mas sim as limitações naturais do Python. O KNN hardcore utiliza laços de repetição nativos que calculam as distâncias por "força bruta" sequencialmente, possuindo alta complexidade temporal. Em contrapartida, o Sklearn é altamente otimizado para produção: ele executa cálculos de distância vetorial em baixo nível (utilizando a linguagem C/Cython via NumPy) e emprega estruturas de dados complexas de indexação espacial (como KD-Trees ou Ball Trees), que reduzem drasticamente o espaço de busca pelos vizinhos.

Dessa forma, fica claro que a implementação manual consome muito mais tempo e memória, sendo inviável para bases de dados massivas, justificando a adoção de bibliotecas consolidadas no mercado.

4. Conclusão

O desenvolvimento do classificador KNN hardcore atingiu plenamente seu objetivo de demonstrar o funcionamento interno do algoritmo. Do ponto de vista lógico e matemático, a implementação manual foi um sucesso absoluto, validado pela exatidão das métricas preditivas (acurácia, precisão e revocação) que se mostraram idênticas ao modelo referencial do Sklearn para todos os valores de *k* avaliados.

No entanto, enquanto a precisão foi garantida, o custo computacional - refletido em um tempo de execução aproximadamente 94 vezes superior - torna a implementação nativa inviável para aplicações reais. Conclui-se que programar modelos do zero é um exercício essencial para o domínio dos fundamentos de Inteligência Artificial, mas a adoção de bibliotecas otimizadas (com suporte a rotinas de baixo nível e estruturas de indexação espacial) é uma exigência técnica inegociável para garantir eficiência de processamento e escalabilidade em ambientes de produção.